```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully!")
```

```
TensorFlow version: 2.20.0
Libraries imported successfully!
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()

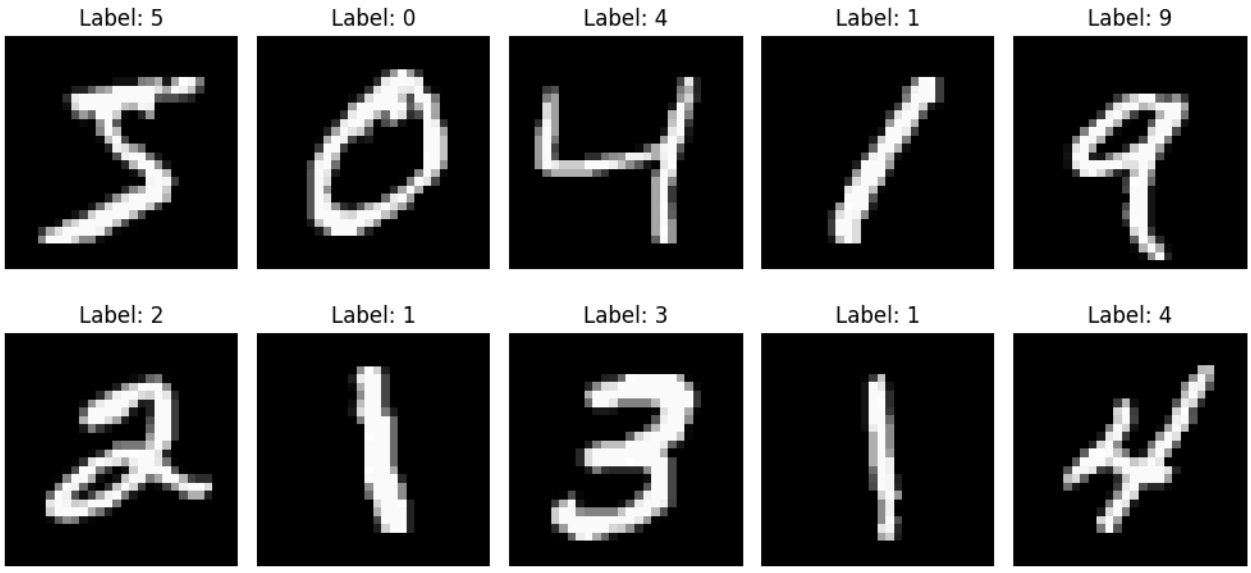
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

```
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

print("New training shape:", x_train.shape)
print("New testing shape:", x_test.shape)
```

```
New training shape: (60000, 28, 28, 1)
New testing shape: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    # Convolution Layer
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    # Pooling Layer
    layers.MaxPooling2D((2, 2)),

    # Second Convolution Layer
    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    # Second Pooling Layer
    layers.MaxPooling2D((2, 2)),

    # Flatten
    layers.Flatten(),

    # Fully Connected Layer
    layers.Dense(128, activation="relu"),

    # Output Layer
    layers.Dense(10, activation="softmax")
])

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserW
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)
```

```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 63ms/step - accuracy: 0.9488 - loss: 0.1685 - val_accuracy: 0
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 52s 62ms/step - accuracy: 0.9842 - loss: 0.0504 - val_accuracy: 0
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 84s 64ms/step - accuracy: 0.9895 - loss: 0.0331 - val_accuracy: 0
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 52s 62ms/step - accuracy: 0.9919 - loss: 0.0252 - val_accuracy: 0
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 59ms/step - accuracy: 0.9937 - loss: 0.0187 - val_accuracy: 0
```

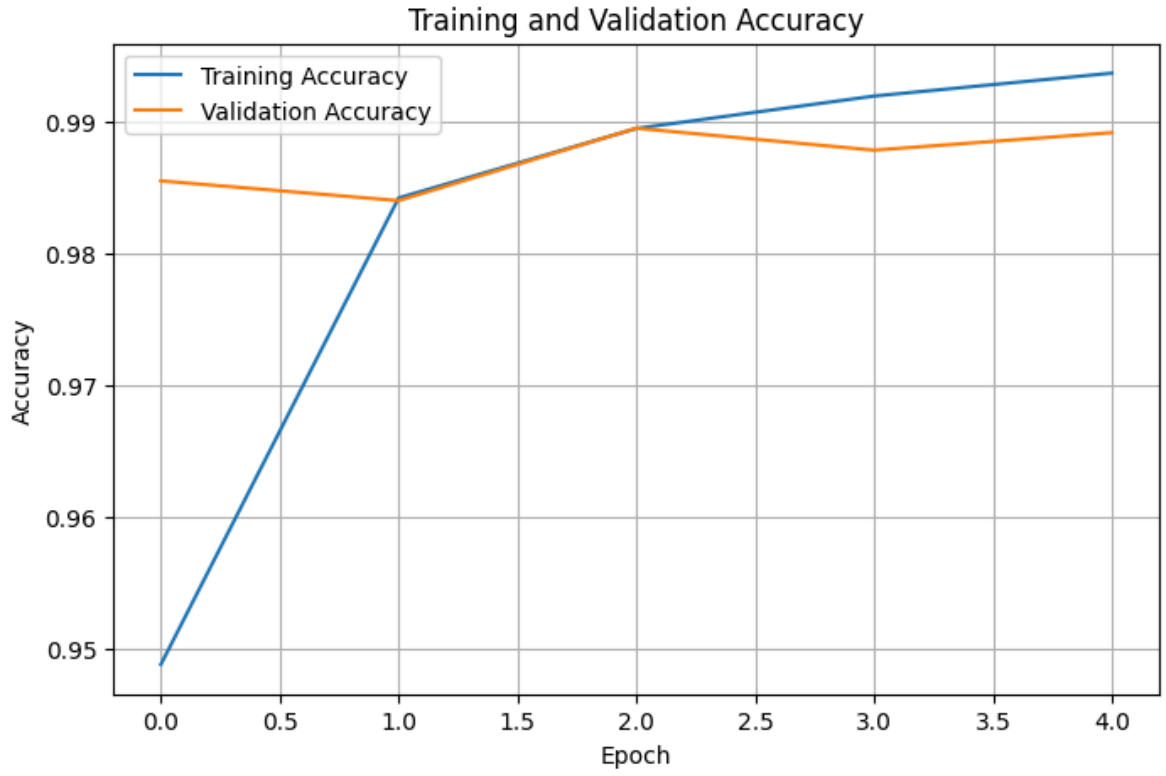
```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
```

```
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid()
plt.show()
```

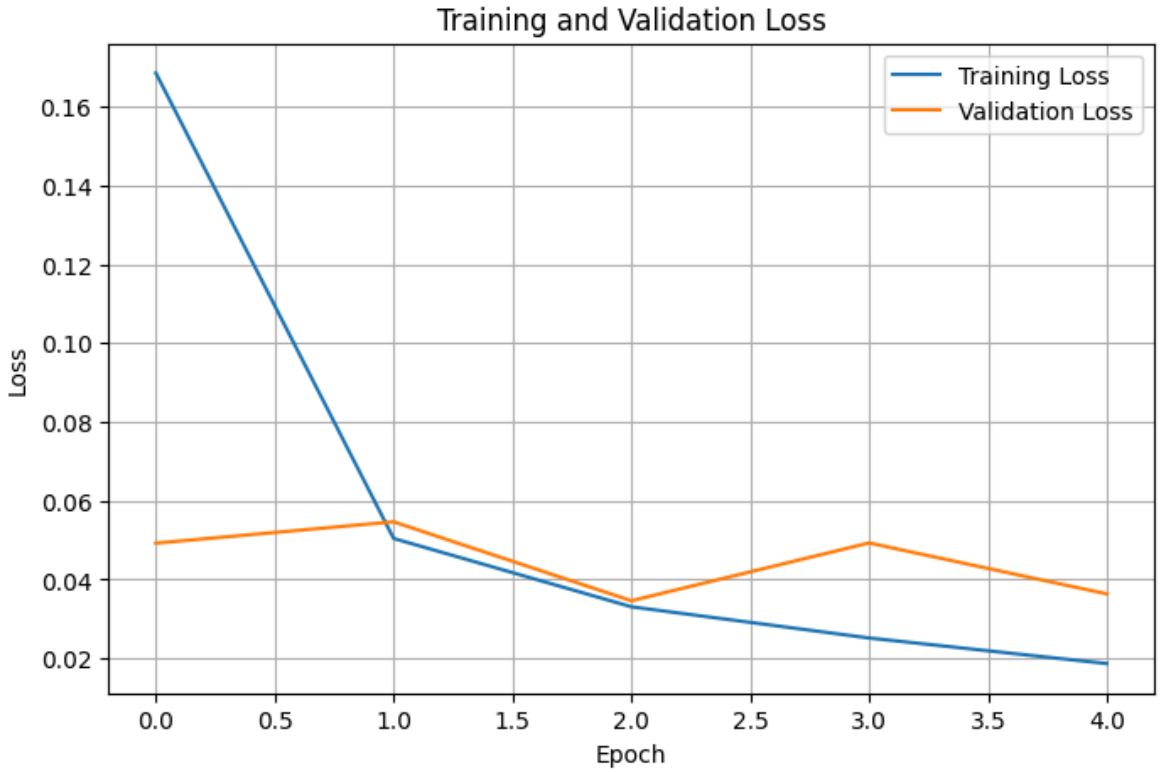



```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid()
plt.show()
```




```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 0.03126059100031853
Test Accuracy: 0.9904000163078308
```

```
predictions = model.predict(x_test)

predicted_labels = np.argmax(
    predictions,
    axis=1
)

print("Predictions generated successfully!")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step
Predictions generated successfully!
```

```
plt.figure(figsize=(12, 8))

for i in range(15):

    plt.subplot(3, 5, i + 1)

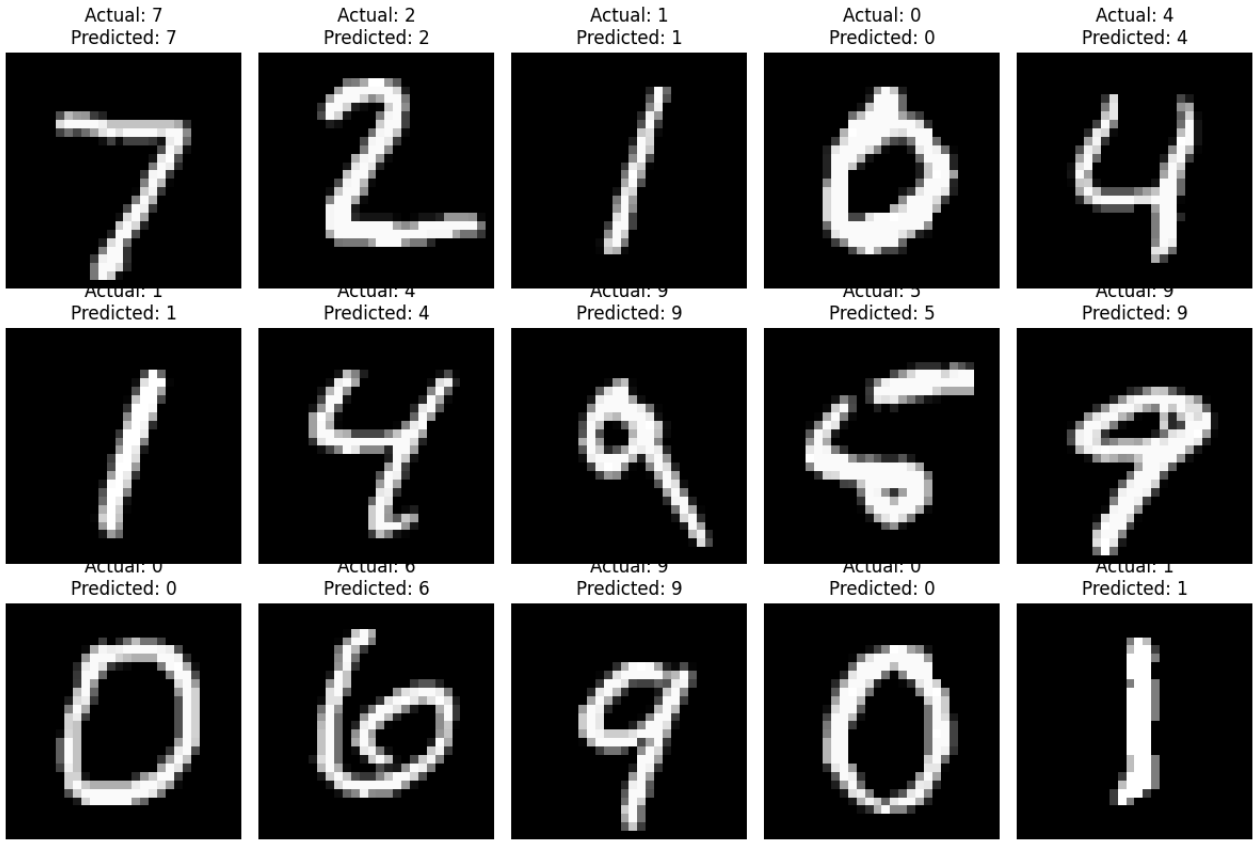
    plt.imshow(
        x_test[i].reshape(28, 28),
        cmap="gray"
```

```
    )

    plt.title(
        f"Actual: {y_test[i]}\n"
        f"Predicted: {predicted_labels[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(
    y_test,
    predicted_labels
)

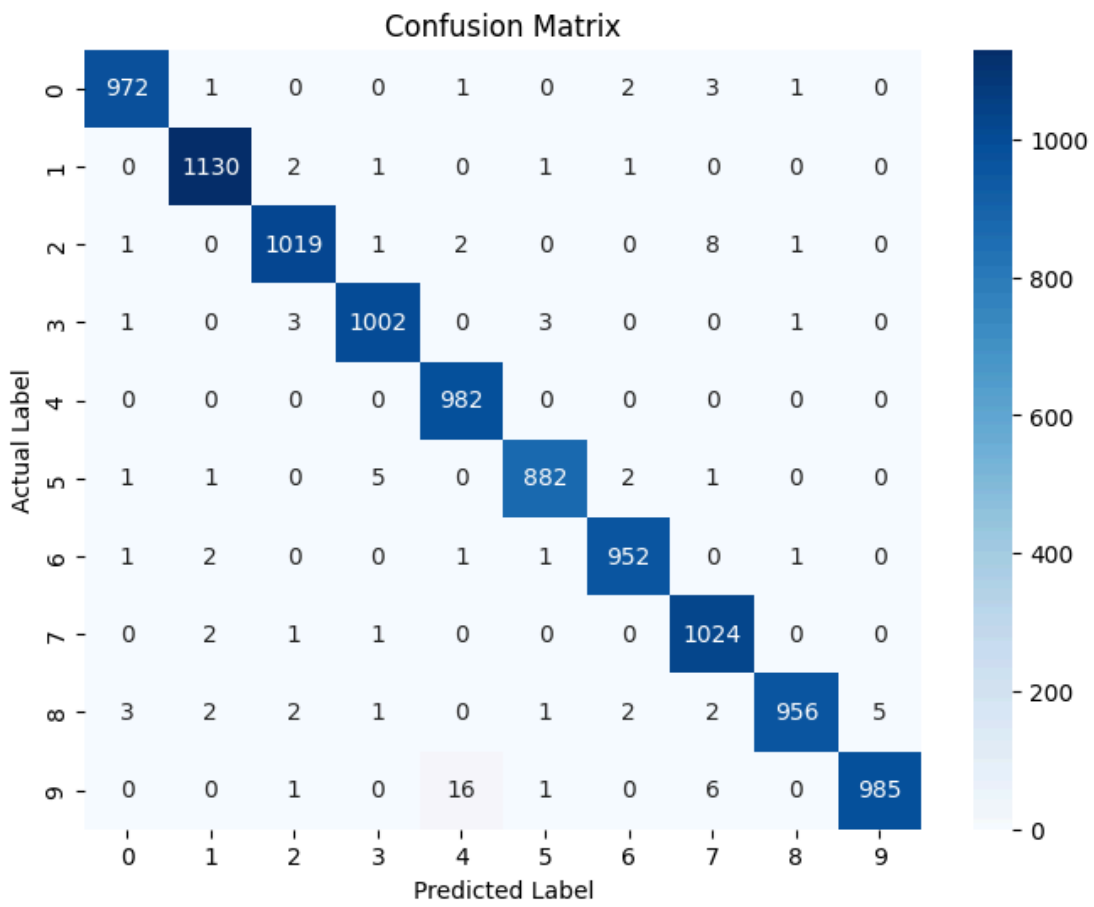
plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
```

```
plt.title("Confusion Matrix")

plt.show()
```




```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        predicted_labels
    )
)
```

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       980
           1       0.99      1.00      0.99      1135
           2       0.99      0.99      0.99      1032
           3       0.99      0.99      0.99      1010
           4       0.98      1.00      0.99       982
           5       0.99      0.99      0.99       892
           6       0.99      0.99      0.99       958
           7       0.98      1.00      0.99      1028
           8       1.00      0.98      0.99       974
           9       0.99      0.98      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```
wrong_predictions = np.where(
    predicted_labels != y_test
)[0]

print(
    "Number of misclassified images:",
    len(wrong_predictions)
)
```

```
Number of misclassified images: 96
```

```
plt.figure(figsize=(12, 8))

for i, index in enumerate(wrong_predictions[:15]):

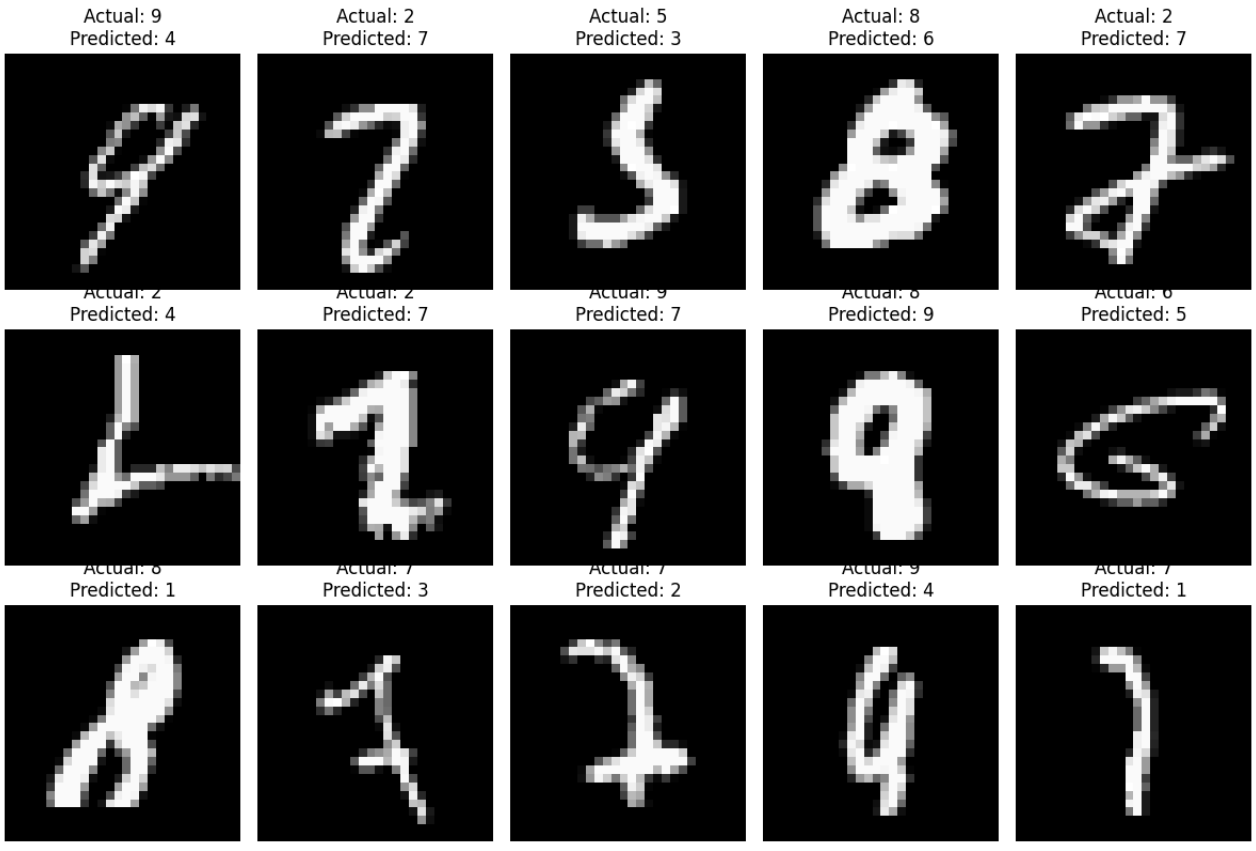
    plt.subplot(3, 5, i + 1)

    plt.imshow(
        x_test[index].reshape(28, 28),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[index]}\n"
        f"Predicted: {predicted_labels[index]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
index = 0

image = x_test[index]

prediction = model.predict(
    image.reshape(1, 28, 28, 1)
)

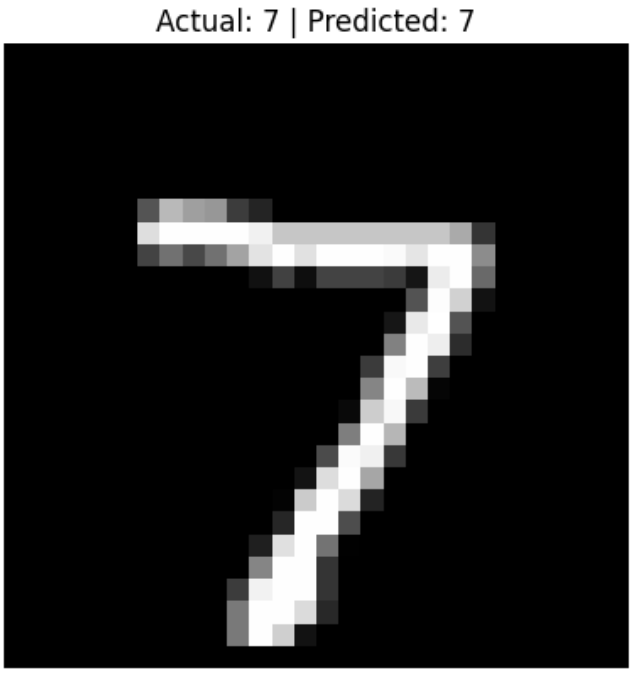
predicted_digit = np.argmax(prediction)

plt.imshow(
    image.reshape(28, 28),
    cmap="gray"
)

plt.title(
    f"Actual: {y_test[index]} | "
    f"Predicted: {predicted_digit}"
)

plt.axis("off")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
```




```
model.save("mnist_cnn_model.h5")

print("Model saved successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.sav
Model saved successfully!
```

```
Start coding or generate with AI.
```